

TP4 — Aprendizaje no supervisado: SOM y k-medias

Inteligencia Computacional · FICH-UNL

Cómo se implementa cada paso del algoritmo, y qué dicen los resultados

*Este apunte acompaña al TP4. `ipynb`. Para cada método: **qué paso del algoritmo es**, **por qué está ahí** y **qué hace cada línea**; después, **qué dijeron los números** de las corridas. Todos los valores citados salen de corridas reales con semilla 2077. Las referencias del tipo «§5» apuntan al apunte de teoría `Teorias/05-MapasOrganizativos/Resumenes/01-mapas-autoorganizativos.md`.*

0. Cómo pensar el SOM antes de escribir una línea

Tres imágenes que conviene tener antes del código, porque todo el resto se deduce de ellas.

El peso no es un coeficiente: es una posición. En el perceptrón, un peso multiplicaba a una entrada. En el SOM **no multiplica nada** — en toda la implementación no aparece ni una vez el producto `pesos * entrada`. Si los datos tienen N columnas, cada neurona tiene N pesos, y esos N números juntos son **un punto de $\mathbb{R}^N$** , el mismo espacio donde viven los patrones. Por eso se pueden dibujar juntos.

La red de pesca. Imaginá una hoja con los 750 puntos de `circulo.csv` dibujados, y 100 chinches clavadas en la misma hoja. El peso de una neurona es **dónde está clavada su chinche**. Entrenar es un solo verbo: **mover chinches**. Se pincha un dato, se busca la chinche más cercana, y se la corre un poco hacia ahí.

Las chinches están cosidas entre sí. No son 100 chinches sueltas: están atadas formando una malla de 10×10 , y esa costura **está puesta de antemano**, antes de ver un solo dato. La chinche 0 está atada a la 1 y a la 10 siempre, aunque hayan quedado lejísimo una de otra. Cuando se mueve una, los hilos arrastran a las vecinas.

OJO — no es el pizarrón del detective

El detective clava fotos y *después* decide cuáles están relacionadas y tira el hilo: el hilo es el resultado de la investigación. El SOM hace lo contrario: **impone las relaciones de entrada** y deja que los datos acomoden las posiciones. Al terminar resulta que los datos que caen en nudos vecinos se parecen, y eso **emergió**, nadie lo puso.

OJO — el SOM no clasifica, agrupa

Al terminar el entrenamiento hay 100 chinches repartidas y **ninguna tiene cartelito**. El SOM nunca vio una etiqueta. Ponerle nombre a cada neurona es un trámite **posterior y supervisado** (§8), y es lo que pide el punto 9 del ejercicio 2.

Los dos espacios son dos estructuras distintas del código

Lo que en el apunte de teoría es «el espacio del mapa» y «el espacio de entrada», acá son dos cosas concretas y separadas:

Apunte	En el código	Cuántos números	¿Se aprende?
Espacio de entrada $\mathbb{R}^N$	<code>self.pesos[j]</code>	tantos como entradas	sí : es lo único que se mueve
Espacio del mapa (rejilla)	<code>coordenadas_en_el_mapa(j)</code>	2 (fila, columna), siempre	no : es fija

En el ejercicio 1 las dos son pares de números y por eso se confunden. En el ejercicio 2 se despegan solas: Iris tiene 4 entradas, así que cada neurona tiene **4 pesos** y la rejilla sigue teniendo **2** coordenadas.

IDEA DE FONDO — para qué sirve todo esto, visto desde el TP

Con el círculo, el mapa parece un adorno: los datos ya eran dibujables. Con Iris recién se ve el punto: $\mathbb{R}^4$ no se puede dibujar, pero la cuadrícula de 10×10 **sí**, siempre, sea cual sea la dimensión de la entrada. Ésa es la razón por la que el enunciado pide el gráfico de frecuencias sólo para el SOM: **k-medias no tiene mapa**. Sus centroides no están en ningún lado, no hay vecindad entre ellos, no hay nada que dibujar más que el espacio de entrada proyectado.

Claves de la sección 0

Clave	Qué tenés que poder responder
Qué es un peso	Un punto de $\mathbb{R}^N$, no un coeficiente. No multiplica nada
Cuántos pesos por neurona	Tantos como entradas: 2 en el ejercicio 1, 4 en el 2
De dónde salen los hilos	De filas y columnas: se fijan antes de entrenar y no cambian
Qué aprende el SOM	Sólo las posiciones. La vecindad es un dato, no un resultado
Qué NO hace	Clasificar. Eso es el etiquetado posterior

1. La clase MapaAutoorganizativo, método por método

Cada método es **un paso del algoritmo** de la §13, para poder señalarlos en la defensa.

Método	Paso	Qué mira	Qué responde
<code>__init__</code>	1 y 2	—	dónde arrancan las chinchas
<code>coordenadas_en_el_mapa</code>	1	la rejilla	dónde está la neurona j en la cuadrícula
<code>neurona_ganadora</code>	4	<code>self.pesos</code>	cuál gana
<code>neuronas_del_entorno</code>	5	la rejilla	quiénes se mueven
<code>adaptar</code>	6	<code>self.pesos</code>	cuánto se mueven
<code>parametros_de_la_epoca</code>	7	el cronograma	con qué η y qué R
<code>entrenar</code>	3 y 8	todo	el bucle

1.1 Inicialización

```
def __init__(self, filas, columnas, cantidad_de_entradas, semilla=2077):
    self.filas = filas
    self.columnas = columnas
    self.cantidad_de_neuronas = filas * columnas
    self.cantidad_de_entradas = cantidad_de_entradas

    generador = np.random.default_rng(semilla)
    self.pesos = generador.uniform(low=-0.5, high=0.5,
                                   size=(self.cantidad_de_neuronas, cantidad_de_entradas))
```

`self.pesos` es una matriz de $M \times N$: una fila por neurona, una columna por entrada. La fila j es el vector $\mathbf{w}_j$.

El rango $[-0.5; +0.5]$ es el que pide la cátedra (§5, paso 1). Arranca con toda la malla apretada cerca del origen: es la maraña previa al despliegue, y es lo que se ve en el primer cuadro del GIF.

OJO — la cuadrícula no está guardada en ningún lado

Las neuronas se guardan como una **lista plana** de M posiciones numeradas de 0 a $M - 1$. La cuadrícula no es un dato: es una **cuenta**, y esa cuenta es el método siguiente. Ésa es la razón por la que el SOM unidimensional del ejercicio 1 sale gratis: `filas=1`, `columnas=100` y **no cambia una línea del algoritmo**.

1.2 De número de neurona a butaca de la cuadrícula

```
def coordenadas_en_el_mapa(self, indice_de_neurona):
    fila = indice_de_neurona // self.columnas
    columna = indice_de_neurona % self.columnas
    return fila, columna
```

Es la cuenta de las butacas del cine. La numeración corrida llena fila por fila:

```
fila 0:  0  1  2  3  4  5  6  7  8  9
fila 1: 10 11 12 13 14 15 16 17 18 19
...
fila 6: 60 61 62 63 64 65 66 67 68 69
```

La **división entera** dice cuántas filas completas pasaron antes ($63 // 10 = 6$) y el **resto** dice cuánto sobró después de llenarlas ($63 \% 10 = 3$). Butaca $63 \rightarrow$ fila 6, columna 3. La vuelta inversa es $\text{índice} = \text{fila} \times C + \text{columna}$.

Se divide por `self.columnas` y no por un 10 escrito a mano: **siempre se divide por el largo de cada fila**. Con 7 columnas, la 63 sería fila 9, columna 0.

OJO — consecutivo en la lista no es vecino en el mapa

La **69** está en (fila 6, columna 9) y la **70** en (fila 7, columna 0): números consecutivos, esquinas opuestas del mapa. **No son vecinas**. Por eso el entorno no se puede calcular sumando y restando índices ($\text{ganadora} \pm 1$, $\text{ganadora} \pm 10$): con esa cuenta la 69 se «envuelve» y termina siendo vecina de la 70. Es el bug clásico de esta implementación.

1.3 Competencia: la ganadora

```
def neurona_ganadora(self, patron):
    diferencias = self.pesos - patron
    distancias_al_cuadrado = np.sum(diferencias ** 2, axis=1)
    indice_de_la_ganadora = np.argmin(distancias_al_cuadrado)
    return int(indice_de_la_ganadora)
```

Fórmula: §2. $G(x) = \arg \min_j \|x - w_j\|$

self.pesos - patron: NumPy estira el patrón y se lo resta a cada fila. Queda (M, N) : el renglón j es el vector que va de la chinche al puntito.

np.sum(..., axis=1): eleva al cuadrado y suma a lo ancho de cada fila, o sea las N coordenadas de cada neurona. Quedan M números.

La distancia es Pitágoras, con tantos términos como entradas:

$$\|x - w_j\| = \sqrt{\sum_{i=1}^N (x_i - w_{ji})^2}$$

No hay **np.sqrt**, y es deliberado: la raíz es creciente, así que la neurona con menor suma de cuadrados es la misma que con menor distancia. Con 750 patrones $\times$ 1000 épocas $\times$ 100 neuronas son 75 millones de raíces que no se calculan. **En el pizarrón la fórmula se escribe con norma**; la omisión es una optimización de implementación y conviene decirlo así.

OJO — la regla del axis: es el eje que desaparece

La matriz de cuadrados es $(100, 2)$. Con $\text{axis}=1$ desaparece el 2 y queda $(100,)$: un número por neurona, que es lo que se busca. Con $\text{axis}=0$ desaparece el 100 y quedan 2 números —el error en x de todas las neuronas sumado—, que no significa nada: argmin devolvería 0 o 1 siempre y el SOM movería siempre la misma chinche. El chequeo es imprimir `.shape`.

argmin devuelve el índice del menor, no el valor: es el arg min literal. Va **mínimo**, no máximo —en el perceptrón ganaba la de mayor activación, acá la de menor distancia.

1.4 Cooperación: el entorno

```
def neuronas_del_entorno(self, indice_de_la_ganadora, radio):
    fila_ganadora, columna_ganadora = self.coordenadas_en_el_mapa(indice_de_la_ganadora)
    indices_del_entorno = []

    for indice_de_neurona in range(self.cantidad_de_neuronas):
        fila, columna = self.coordenadas_en_el_mapa(indice_de_neurona)

        distancia_en_filas = abs(fila - fila_ganadora)
        distancia_en_columnas = abs(columna - columna_ganadora)
        distancia_en_el_mapa = max(distancia_en_filas, distancia_en_columnas)

        if distancia_en_el_mapa <= radio:
            indices_del_entorno.append(indice_de_neurona)

    return indices_del_entorno
```

Fórmula: §3. $\Lambda_G(n) = \{j : \max(|\Delta_{\text{fila}}|, |\Delta_{\text{col}}|) \leq R(n)\}$

Entra un índice y sale una lista de índices, pero la decisión se toma en la rejilla. Fijate que en todo el método no aparece la palabra pesos: este método no sabe dónde está clavada ninguna chinche.

El max es lo que hace el entorno **cuadrado** (distancia de Chebyshev): alcanza con que la peor de las dos diferencias entre en el radio, así que la diagonal cuenta como vecina. Con la distancia común quedaría un círculo y con radio 1 se caerían las cuatro esquinas: 5 neuronas en vez de 9.

El $\leq$ incluye a la ganadora, que está a distancia 0 de sí misma. Tiene que estar en la lista: es la que más debe moverse en su zona.

El recorte en los bordes sale gratis. Como se recorren sólo las neuronas que existen, las que se saldrían de la cuadrícula simplemente no aparecen. Verificado:

Ganadora	Posición	Entorno con $R = 1$	Cuántas
63	interior	52, 53, 54, 62, 63, 64, 72, 73, 74	9
60	borde izquierdo	50, 51, 60, 61, 70, 71	6
69	borde derecho	58, 59, 68, 69, 78, 79	6
0	esquina	0, 1, 10, 11	4

Ni un if sobre los bordes. Y fíjate que en el entorno de la **69 no está la 70**.

Dos casos que salen del mismo código sin tocar nada: con **radio 0** devuelve sólo la ganadora (la etapa 3, y también el modo « k -medias» del experimento de control), y con el mapa de 1×100 el entorno se vuelve un segmento de la recta: [62, 63, 64].

Control contra el apunte: con el mapa de 3×3 del ejemplo numérico de la §14, el método devuelve {2, 3, 5, 6} para la esquina y las 9 para el centro — idéntico a lo que está escrito a mano ahí (corrigiendo que el apunte numera desde 1 y el código desde 0).

1.5 Adaptación

```
def adaptar(self, patron, indices_del_entorno, velocidad_de_aprendizaje):
    for indice_de_neurona in indices_del_entorno:
        peso_actual = self.pesos[indice_de_neurona]
        desplazamiento = velocidad_de_aprendizaje * (patron - peso_actual)
        self.pesos[indice_de_neurona] = peso_actual + desplazamiento
```

Fórmula: §5. $\mathbf{w}_j \leftarrow \mathbf{w}_j + \eta (\mathbf{x} - \mathbf{w}_j)$

patron - peso_actual es el vector que va **de la chinche al puntito**. El **orden importa**: al revés, la chinche se aleja del dato y el mapa explota en vez de ordenarse.

PARA LA DEFENSA — dónde está la segunda rama de la ecuación

El apunte escribe la adaptación en dos casos, y el segundo dice que las neuronas fuera del entorno **no se tocan**. En el código esa rama **no está escrita**: el bucle recorre únicamente indices_del_entorno. Conviene decirlo explícito —«la segunda rama está implícita en que sólo itero sobre el entorno»—, porque es el punto que se busca. Sin ella, el desarrollo es k -medias.

1.6 El cronograma de las tres etapas

```
def parametros_de_la_epoca(self, epoca, epocas_totales):
    radio_inicial = max(self.filas, self.columnas) // 2
    avance = epoca / epocas_totales

    if avance < 0.20:
        proporcion_de_la_etapa = avance / 0.20
        velocidad_de_aprendizaje = 0.9 - proporcion_de_la_etapa * (0.9 - 0.7)
        radio = radio_inicial
        nombre_de_la_etapa = "ordenamiento"

    elif avance < 0.40:
        proporcion_de_la_etapa = (avance - 0.20) / 0.20
        velocidad_de_aprendizaje = 0.7 - proporcion_de_la_etapa * (0.7 - 0.1)
        tramo = int(proporcion_de_la_etapa * radio_inicial)
        radio = max(1, radio_inicial - tramo)
        nombre_de_la_etapa = "transicion"

    else:
        velocidad_de_aprendizaje = 0.01
        radio = 0
        nombre_de_la_etapa = "ajuste fino"

    return velocidad_de_aprendizaje, radio, nombre_de_la_etapa
```

avance es el reloj general, de 0 a 1. Trabajar con la proporción y no con el número de época hace que el cronograma funcione igual con 200 o con 1000 épocas.

proporcion_de_la_etapa es el mismo reloj **dentro** de cada etapa, y por eso se calcula distinto en cada rama: en la segunda hay que restar primero el 0,20 que ya pasó, para que arranque en 0.

La **interpolación** tiene siempre la misma forma — se arranca en el valor inicial y se descuenta la parte del camino recorrida:

$$\text{valor} = \text{inicio} - p \cdot (\text{inicio} - \text{fin})$$

tramo existe porque el radio es un número **entero** de casilleros: no puede bajar suave, tiene que ir 5, 4, 3, 2, 1. La etapa se parte en tantos escalones como valores haya que recorrer, e `int()` dice en cuál se está. Es el reparto del ejemplo aritmético de la clase, calculado en vez de escrito a mano. El `max(1, ...)` frena en 1: el radio 0 corresponde a la etapa siguiente.

Cómo queda, con 1000 épocas y mapa de 10×10 :

Época	η	Radio	Etapas
0	0,900	5	ordenamiento
190	0,710	5	ordenamiento
200	0,700	5	transición
280	0,460	3	transición
399	0,103	1	transición
400	0,010	0	ajuste fino

η pasa **suave** de la etapa 1 a la 2 (termina y arranca en 0,7) y pega un **salto** al entrar a la 3, porque el apunte pide η constante en el ajuste fino. En la transición, cada valor del radio dura unas 40 épocas (200 repartidas en 5 escalones).

PARA LA DEFENSA — qué es de la cátedra y qué es decisión propia

De la cátedra (§6): las tres etapas y sus nombres; Λ_G inicial $\approx$ medio mapa; η entre 0,9 y 0,7 en la primera; $\Lambda_G \rightarrow 1$ y $\eta \rightarrow 0,1$ en la segunda; $\Lambda_G = 0$ y η constante entre 0,1 y 0,01 en la tercera; el reparto en escalones de la transición. **Decisión propia:** el reparto 20 / 20 / 60. Las duraciones del apunte suman unas 5000 épocas y el enunciado limita a 1000, así que hubo que comprimir manteniendo las proporciones. También son elecciones dentro de lo que la cátedra deja abierto: que η baje en rampa de 0,9 a 0,7 en la etapa 1, que la interpolación sea lineal y no exponencial, y tomar 0,01 del rango de la etapa 3. La frase que cubre el punto: «las tres etapas y los valores son los de la cátedra; lo que ajusté es el reparto de épocas, porque el enunciado limita a 1000 y las duraciones del apunte suman unas 5000». Decir que el 20/20/60 está en el apunte **no** es defendible: no está.

1.7 El bucle

```
def entrenar(self, patrones, epocas_totales=1000, epocas_entre_cuadros=10, semilla_del_orden=2077):
    generador = np.random.default_rng(semilla_del_orden)
    historial_de_pesos = []
    historial_de_etiquetas = []

    for epoca in range(epocas_totales):
        velocidad_de_aprendizaje, radio, nombre_de_la_etapa = self.parametros_de_la_epoca(epoca, epocas_totales)

        if epoca % epocas_entre_cuadros == 0:
            historial_de_pesos.append(self.pesos.copy())
            historial_de_etiquetas.append(
                f"epoca {epoca} | {nombre_de_la_etapa} | radio {radio} | eta {velocidad_de_aprendizaje:.3f}")

        orden_de_los_patrones = generador.permutation(len(patrones))

        for indice_de_patron in orden_de_los_patrones:
            patron = patrones[indice_de_patron]
            indice_de_la_ganadora = self.neurona_ganadora(patron)
            indices_del_entorno = self.neuronas_del_entorno(indice_de_la_ganadora, radio)
            self.adaptar(patron, indices_del_entorno, velocidad_de_aprendizaje)

        historial_de_pesos.append(self.pesos.copy())
        historial_de_etiquetas.append(f"epoca {epocas_totales} | final")

    return historial_de_pesos, historial_de_etiquetas
```

El corazón son los **tres renglones del final**, en ese orden: quién gana → quiénes la acompañan → moverlos. Todo lo demás es andamiaje.

Los parámetros se piden una vez por época, no por patrón: es el paso 7 de la §13, η y R cambian con las épocas y dentro de una época quedan fijos. Por eso la llamada está fuera del bucle interno.

permutation baraja el orden de presentación en cada época (§13, paso 3). Con orden fijo el mapa se sesga por el orden del archivo.

OJO — el `.copy()` no es opcional

`self.pesos` se modifica en el lugar todo el tiempo. Guardar `self.pesos` sin copiar deja el historial lleno de **referencias al mismo array**: el GIF saldría con 100 cuadros idénticos, todos con el estado final.

1.8 Los dos métodos de apoyo

```
def aristas_de_la_malla(self):
    pares_de_vecinas = []
    for indice_de_neurona in range(self.cantidad_de_neuronas):
        fila, columna = self.coordenadas_en_el_mapa(indice_de_neurona)
        if columna < self.columnas - 1:
            pares_de_vecinas.append((indice_de_neurona, indice_de_neurona + 1))
        if fila < self.filas - 1:
            pares_de_vecinas.append((indice_de_neurona, indice_de_neurona + self.columnas))
    return pares_de_vecinas

def ganadoras_de_todos(self, patrones):
    return np.array([self.neurona_ganadora(patron) for patron in patrones])
```

`aristas_de_la_malla` devuelve qué pares hay que unir con una línea: cada neurona con la de su derecha y con la de abajo, y los `if` evitan salirse del borde. Es lo que pide el enunciado con «*grafique líneas de unión entre pares de neuronas vecinas*» — las líneas son de la **rejilla**, dibujadas sobre las **posiciones** del espacio de entrada.

`ganadoras_de_todos` es el **modo de uso** de la red: un SOM entrenado, ante un dato, lo único que hace es el paso 2.

IDEA DE FONDO — el paso 2 es la red funcionando; el 3 y el 4 sólo existen mientras aprende

En el perceptrón la salida era un número que salía de una cuenta. Acá la salida es un **nombre**: «ganó la 63». La red no calcula un valor, señala un ganador.

Claves de la sección 1

Clave	Qué tenés que poder responder
// y %	Traducen índice a fila y columna; la cuadrícula es una cuenta, no un dato
Por qué sin <code>sqrt</code>	La raíz es creciente: no cambia el <code>arg min</code>
<code>axis=1</code>	Es el eje que desaparece: queda un número por neurona
Dónde está la 2.ª rama	Implícita: el bucle de adaptar sólo recorre el entorno
<code>9 / 6 / 4</code>	Interior, borde y esquina con $R = 1$, sin ningún <code>if</code>
Por qué no <code>ganadora±1</code>	La 69 y la 70 son consecutivas y no son vecinas
El <code>.copy()</code>	Sin él, el historial son 100 referencias al mismo array

2. Ejercicio 1 — Resultados

Todas las corridas con semilla 2077, 1000 épocas y el mismo cronograma. La fila **sin entorno** es un control: el mismo código forzando $\Lambda_G = 0$ en todas las etapas, que es k -medias en línea.

Corrida	Cruces de la malla	Neuronas usadas	Error de cuantización	Dist. vecinas	Dist. par cualquiera
Círculo 10 × 10	0	100/100	0,0573	0,187	0,893
T 10 × 10	3	95/100	0,0335	0,177	0,834
T 1 × 100	0	99/100	0,0289	0,114	0,900
Círculo sin entorno	3742	100/100	0,0536	0,903	0,908
T sin entorno	3794	79/100	0,0347	0,907	0,912

Las tres métricas, definidas:

- **Cruces de la malla:** cuántos pares de aristas de la rejilla se cortan entre sí al dibujarlas sobre las posiciones de los pesos. Cero significa red desplegada sin pliegues.
- **Error de cuantización:** $\frac{1}{L} \sum_{\ell} \|\mathbf{x}_{\ell} - \mathbf{w}_{G(\mathbf{x}_{\ell})}\|$, o sea qué tan bien representa el mapa a los datos. Es lo que el SOM comparte con k -medias.
- **Distancia vecinas vs par cualquiera:** $\|\mathbf{w}_a - \mathbf{w}_b\|$ promedio entre neuronas vecinas **en el mapa**, contra la misma distancia entre dos neuronas cualesquiera. Es la medida directa del ordenamiento.

1. El ordenamiento topológico se consiguió, y hay un número. 0 cruces con entorno contra 3742 sin entorno, mismos datos y misma inicialización. Lo único que cambió fue el radio.

2. El entorno no mejora el agrupamiento, sólo lo ordena. El error de cuantización es prácticamente igual en los dos casos (0,0573 con entorno contra 0,0536 sin), e incluso **levemente mejor sin él**: liberada de arrastrar vecinas, cada neurona se planta más exactamente en el centro de su región. El entorno cuesta un poco de error de cuantización y compra todo el orden. Es exactamente lo que dice la §7 del apunte de teoría.

3. La comparación vecinas / par cualquiera es la prueba del orden. Con entorno, dos vecinas de la rejilla están a 0,187 y dos cualesquiera a 0,893: **cinco veces más cerca**. Sin entorno, 0,903 contra 0,908 — ser vecinas en el mapa **no dice nada** sobre la posición en el espacio de entrada.

4. La T es más exigente que el círculo por no ser convexa. 3 cruces y 5 neuronas sin usar, contra 0 y 0. La rejilla es rígida y no se puede romper: para cubrir el palo y la barra tiene que estirar aristas sobre el hueco, y las neuronas que quedan en esa zona vacía no ganan ningún patrón.

5. El entorno rescata neuronas mal inicializadas. En la T, el control sin entorno deja 79 de 100 neuronas vivas contra 95 con entorno. Sin entorno, un prototipo que arranca lejos de la nube no gana nunca y nunca se corrige.

6. El SOM unidimensional cuantiza mejor y conserva menos. Sobre la T, la tira alcanza 0,0289 contra 0,0335 del mapa bidimensional: no está atada a una grilla y se acomoda con más libertad. Eso mismo es lo que pierde — conserva la vecindad en **una sola dirección**, y como serpentea para cubrir una región del plano, dos patrones cercanos pueden activar neuronas muy separadas del mapa.

7. Costo. 1000 épocas sobre 750 patrones con mapa de 10×10 : **unos 70 segundos**. Por etapa: 0,139 s/época con radio 5, 0,092 con radio 3 y 0,039 con radio 0 — el costo baja con el radio porque adaptar mueve 81 neuronas por patrón al principio y una sola al final.

PARA LA DEFENSA — el número que conviene tener a mano

«Con entorno, 0 cruces; sin entorno, 3742. El error de cuantización es casi el mismo. O sea que el entorno no agrupa mejor: **ordena**.»

3. k -medias

3.1 Qué es, en dos pasos

Se le pide partir los datos en k grupos. Son dos movimientos que se alternan hasta que nadie se mueve:

1. **Asignar.** Cada dato se anota con el centro más cercano.
2. **Recalcular.** Cada centro se muda al **promedio de los datos que se le anotaron**.

Y se vuelve al 1: al moverse los centros, algunos datos cambian de dueño. Cuando una vuelta entera no cambia a nadie, terminó.

$$\text{paso 1: } c(\mathbf{x}) = \arg \min_i \|\mathbf{x} - \mathbf{m}_i\| \quad \text{paso 2: } \mathbf{m}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x}$$

En chinchas: clavás 3 chinchas donde sea; cada puntito levanta la mano por la que tiene más cerca; cada chinche se muda al centro de gravedad de su manada; repetís.

IDEA DE FONDO — el SOM es k -medias con hilos

El paso 1 de k -medias es la competencia del SOM: misma fórmula, mismo código. Lo que cambia es el paso 2.

	k -medias	SOM
Elegir a quién le toca el dato	distancia mínima al centroide	idéntico
Cómo se mueve el ganador	salta al promedio de su manada	se acerca un η al dato
Cuándo se mueve	después de ver todos los datos (por lotes)	con cada dato (en línea)
¿Arrastra a otros?	no	sí: el entorno en la rejilla
Cuándo termina	solo, cuando nadie cambia	cuando se acaban las épocas
Resultado	k grupos sin relación entre sí	grupos ordenados en un mapa

No hay velocidad de aprendizaje en k -medias: el centroide **salta** al promedio. Por eso converge en pocas vueltas (con Iris, entre 1 y 6) contra las 1000 épocas del SOM. Y por eso tampoco hay etapas ni entorno que programar.

3.2 La clase KMedias, método por método

class KMedias:

```
def __init__(self, cantidad_de_grupos, semilla=2077):
    self.cantidad_de_grupos = cantidad_de_grupos
    self.semilla = semilla
    self.centroides = None

def inicializar_centroides(self, patrones):
    generador = np.random.default_rng(self.semilla)
    indices_elegidos = generador.choice(len(patrones),
                                        size=self.cantidad_de_grupos,
                                        replace=False)
    self.centroides = patrones[indices_elegidos]
```

La inicialización es distinta a la del SOM, y a propósito. Acá se eligen k **patrones del propio conjunto** como centros de arranque — la otra opción que menciona el apunte en §5. La razón: k -medias **no tiene entorno**, así que un centro que arranca lejos de todo no gana nunca, nadie lo arrastra y se queda ahí para siempre; quedarían $k-1$ grupos reales. Arrancando sobre patrones existentes, cada centro tiene al menos un dato suyo garantizado.

Es la conclusión 5 del ejercicio 1 vista del otro lado: sin entorno, las neuronas mal inicializadas mueren.

`replace=False` evita elegir dos veces el mismo patrón, que dejaría dos centros superpuestos y un grupo vacío. **No hace falta** `.copy()`: indexar con un array de índices ya devuelve una copia en NumPy (sería distinto con un slice, que es una vista).

```
def asignar_grupos(self, patrones):
    grupos = np.zeros(len(patrones), dtype=int)

    for indice_de_patron in range(len(patrones)):
        patron = patrones[indice_de_patron]
        diferencias = self.centroides - patron
        distancias_al_cuadrado = np.sum(diferencias ** 2, axis=1)
        grupos[indice_de_patron] = int(np.argmin(distancias_al_cuadrado))

    return grupos
```

Es neurona_ganadora repetida para todos: las tres líneas del medio son idénticas, con `self.centroides` en vez de `self.pesos`. Devuelve un vector con el grupo de cada patrón. El `dtype=int` importa: sin él los grupos quedarían como 0.0, 1.0 y no servirían para indexar ni para la matriz de contingencia.

```
def recalcular_centroides(self, patrones, grupos):
    centroides_nuevos = self.centroides.copy()

    for indice_de_grupo in range(self.cantidad_de_grupos):
        patrones_del_grupo = patrones[grupos == indice_de_grupo]
        if len(patrones_del_grupo) > 0:
            centroides_nuevos[indice_de_grupo] = patrones_del_grupo.mean(axis=0)

    return centroides_nuevos
```

Con seis patrones a mano, para fijar qué hace cada línea. Seis puntos en 2D, $k = 2$, con esta asignación ya hecha:

```
patrón 0: (1, 2) -> grupo 0      patrón 1: (2, 1) -> grupo 1
patrón 2: (0, 4) -> grupo 0      patrón 3: (3, 0) -> grupo 1
patrón 4: (2, 3) -> grupo 0      patrón 5: (4, 2) -> grupo 1
```

o sea `grupos = [0, 1, 0, 1, 0, 1]`. En la vuelta del grupo 0:

`grupos == 0` da la máscara `[True, False, True, False, True, False]`.

`patrones[máscara]` se queda con esas filas:

$$\begin{pmatrix} 1 & 2 \\ 0 & 4 \\ 2 & 3 \end{pmatrix}$$

`.mean(axis=0)` promedia **hacia abajo**, columna por columna:

$$x = \frac{1 + 0 + 2}{3} = 1,0 \quad y = \frac{2 + 4 + 3}{3} = 3,0$$

y el centroide 0 pasa a ser $(1,0; 3,0)$, el **centro de gravedad** de su manada. El grupo 1 da $(3,0; 1,0)$. Con Iris es lo mismo con 4 columnas: `.mean(axis=0)` da las 4 medias.

OJO — acá el `axis` va al revés que en las distancias

Vale la misma regla —es el eje que desaparece— pero cambia qué se busca. En `asignar_grupos` se quería **un número por neurona**, y desaparecían las coordenadas: `axis=1`. Acá se quiere **un número por coordenada**, y desaparecen los patrones: `axis=0`.

El `if len(...) > 0` cubre los grupos vacíos: el promedio de un array vacío da `nan`, ese centroide quedaría en `nan`, su distancia a todo daría `nan` y no ganaría nunca más. Con el `if`, el centro huérfano **se queda donde está** —heredado del `.copy()` inicial— y tiene otra chance. Ése es el motivo de arrancar copiando y no con ceros: con ceros, el huérfano se mudaría al origen, que puede ser un lugar sin ningún dato.

Y ahí está el `.copy()` que **sí** hace falta: entrenar necesita los centroides viejos intactos para compararlos con los nuevos.

```
def entrenar(self, patrones, maximo_de_iteraciones=100):
    self.inicializar_centroides(patrones)
    grupos = self.asignar_grupos(patrones)
    iteraciones_realizadas = 0

    for iteracion in range(maximo_de_iteraciones):
        self.centroides = self.recalcular_centroides(patrones, grupos)
        grupos_nuevos = self.asignar_grupos(patrones)

        cantidad_de_cambios = np.sum(grupos_nuevos != grupos)
        grupos = grupos_nuevos
        iteraciones_realizadas = iteracion + 1

        if cantidad_de_cambios == 0:
            break

    return grupos, iteraciones_realizadas

def inercia(self, patrones, grupos):
    total = 0.0
    for indice_de_patron in range(len(patrones)):
        centroide = self.centroides[grupos[indice_de_patron]]
        diferencia = patrones[indice_de_patron] - centroide
        total += np.sum(diferencia ** 2)
    return total
```

La asignación **antes** del bucle es la que le da a `recalcular_centroides` con qué trabajar en la primera vuelta.

`np.sum(grupos_nuevos != grupos)` cuenta cuántos patrones cambiaron de grupo: la comparación da un vector de `True/False` y `True` vale 1.

PARA LA DEFENSA — el criterio de corte es la diferencia de fondo con el SOM

Si nadie cambió de grupo, la próxima vuelta calcularía los mismos promedios y los mismos grupos: es un punto fijo. **El algoritmo se detiene solo**. El `maximo_de_iteraciones=100` es una red de seguridad defensiva, no el criterio real. En el SOM, en cambio, la cantidad de épocas la fijás vos de antemano.

La **inercia** es lo que k -medias minimiza: $\sum_{\ell} \|\mathbf{x}_{\ell} - \mathbf{m}_{c(\ell)}\|^2$. Se usa dos veces: para comparar corridas con distinta inicialización, y en el ejercicio 3 para la curva del codo.

Claves de la sección 3

Clave	Qué tenés que poder responder
Los dos pasos	Asignar / recalcular, alternados
Qué comparte con el SOM	El paso de asignación: misma fórmula, mismo código
Qué no tiene	Entorno, η , épocas, mapa
Por qué inicializa sobre patrones	Sin entorno, un centro mal ubicado no se rescata nunca
Criterio de corte	Nadie cambia de grupo: converge solo
axis=0 en el promedio	Desaparecen los patrones, queda una media por variable

4. Matrices de contingencia

4.1 El problema que resuelven

Hay dos agrupamientos de los mismos 148 patrones: el de k -medias y el del SOM. La pregunta es si coinciden. Pero **los números de grupo no se pueden comparar directamente**: lo que k -medias llama «grupo 0» el SOM puede llamarlo «neurona 2», y eso no es un desacuerdo — cada método numeró según cómo se inicializó.

La prueba está en el barrido de semillas de la sección 5.3: la misma solución aparece como 60/55/33, 55/60/33 y 33/60/55 según la semilla. **Los nombres son arbitrarios; la partición es la misma.**

La salida es **contar cruces**: cuántos patrones cayeron a la vez en el grupo i de uno y en el grupo j del otro. Eso no depende de cómo se llame cada grupo.

4.2 Un ejemplo de seis patrones

patrón:	0	1	2	3	4	5
k-medias:	0	0	1	1	2	2
SOM:	2	2	0	0	1	1

La tabla queda:

	SOM 0	SOM 1	SOM 2
k-medias 0	0	0	2
k-medias 1	2	0	0
k-medias 2	0	2	0

Los dos métodos coinciden perfectamente, aunque ningún número coincida: cada fila tiene un solo valor distinto de cero. Todos los patrones que uno puso juntos, el otro también.

La regla de lectura: fila con **un solo valor grande** → acuerdo; fila **repartida** → el primer método juntó lo que el segundo separó; **columna repartida** → al revés.

4.3 La implementación

```
def matriz_de_contingencia(agrupamiento_a, agrupamiento_b, cantidad_a=None, cantidad_b=None):
    if cantidad_a is None:
        cantidad_a = int(agrupamiento_a.max()) + 1
    if cantidad_b is None:
        cantidad_b = int(agrupamiento_b.max()) + 1

    tabla = np.zeros((cantidad_a, cantidad_b), dtype=int)

    for indice_de_patron in range(len(agrupamiento_a)):
        fila = agrupamiento_a[indice_de_patron]
        columna = agrupamiento_b[indice_de_patron]
        tabla[fila, columna] += 1

    return tabla
```

El bucle es literalmente «por cada patrón, fijate en qué grupo lo puso cada método y sumale uno a esa celda».

Los parámetros `cantidad_a` y `cantidad_b` cubren un caso real: si una **neurona no ganó nunca**, su número no aparece en el vector y `max()` devuelve uno menos, así que la tabla quedaría con una fila de menos y **no te enterarías**. Pasando la cantidad a mano, la fila vacía aparece — que es justamente lo que se quiere ver.

Se puede verificar contra `contingency_matrix` de `sklearn.metrics.cluster`, que hace exactamente esto.

OJO — no es una matriz de confusión

Se parece, pero no lo es. La matriz de confusión compara **predicción contra verdad** en un problema supervisado, y su diagonal son los aciertos. Acá ninguno de los dos métodos intentó predecir nada, y **la diagonal no significa nada**, porque los números de grupo son arbitrarios. Lo que se mira es si cada fila se concentra en una sola columna, esté donde esté.

5. Ejercicio 2 — Resultados

Datos: `iris81_trn.csv` + `iris81_tst.csv` juntos, **148 patrones**, 4 entradas. Las 3 columnas de salida $\{-1, +1\}$ se convierten a un número de clase con `np.argmax(..., axis=1)`, que devuelve la posición del $+1$. Clases: 45 / 43 / 60.

OJO — por qué no se separa entrenamiento y prueba

La partición `train/test` existe para medir si un modelo **generaliza a datos no vistos**. En clustering no hay nada que predecir: ninguno de los dos métodos ve las etiquetas, no hay acierto que inflar y no hay memorización posible. La pregunta no es *¿generaliza?* sino *¿qué estructura tienen estos 148 patrones?*, y para eso se quiere el conjunto entero. Los archivos están partidos por herencia del TP2, donde sí hacía falta. Las clases de referencia se cargan pero **no entran al entrenamiento**: se usan sólo para comparar al final.

5.1 Las tres contingencias

Con $k = 3$, SOM de 2×2 y semilla 2077, sobre los datos **sin normalizar**:

k -medias contra las clases de referencia

	clase 0	clase 1	clase 2
k-medias 0	0	0	22
k-medias 1	45	43	0
k-medias 2	0	0	38

Mal: el grupo 1 se comió **las clases 0 y 1 enteras** (88 patrones en una bolsa) y la clase 2 la **partió en dos** (22 + 38). Fusionó dos especies y rompió una tercera.

SOM contra las clases de referencia

	clase 0	clase 1	clase 2
neurona 0	26	0	0
neurona 1	0	0	60
neurona 2	19	32	0
neurona 3	0	11	0

Mucho mejor: la neurona 1 se quedó con **los 60 patrones de la clase 2 y con nada más** — pureza perfecta. Las clases 0 y 1, que se solapan, se reparten en las otras tres, y la única mezcla real está en la neurona 2.

k -medias contra el SOM

	neurona 0	neurona 1	neurona 2	neurona 3
k-medias 0	0	22	0	0
k-medias 1	26	0	51	11
k-medias 2	0	38	0	0

Cortaron al revés. Los grupos 0 y 2 de k -medias caen los dos dentro de la neurona 1: lo que k -medias separó, el SOM lo dejó junto. Y el grupo 1 se reparte en las otras tres: lo que k -medias juntó, el SOM lo separó. No discrepan en los bordes — discrepan en **dónde pasa el corte principal**.

5.2 Normalizar no cambia nada

Se repitió todo con los datos llevados a $[-1; +1]$ por columna, $x'_i = 2 \frac{x_i - \min_i}{\max_i - \min_i} - 1$, misma semilla:

	k -medias crudo	k -medias normalizado	SOM crudo	SOM normalizado
Tamaños	22 / 88 / 38	idénticos	26 / 60 / 51 / 11	24 / 60 / 52 / 12
Estructura	clases 0+1 juntas, 2 partida	igual	clase 2 pura en la neurona 1	igual

k -medias dio exactamente los mismos números. El SOM cambió tres o cuatro patrones de lugar. Dos conclusiones:

1. **El mal resultado de k -medias no era por la escala.** La hipótesis era que el largo de pétalo (rango 5,8) pesaba el doble que el ancho de sépalo (rango 2,4) y deformaba las distancias. Si fuera así, emparejar los rangos habría cambiado algo. No cambió nada.
2. **Las clases 0 y 1 se solapan de verdad.** La mezcla de la neurona 2 no es un defecto del método ni de la escala: esas dos especies están pegadas en el espacio de medidas y **no hay hueco donde cortar**.

Que el SOM cambie un poquito y k -medias nada tiene sentido: al SOM la normalización sí le cambia algo, porque sus pesos arrancan en $[-0,5; 0,5]$ y los datos crudos van de 0,1 a 7,9 — sin normalizar, las neuronas arrancan **fuera de la nube**. Que aun así llegue casi al mismo resultado dice que la etapa de ordenamiento hace bien su trabajo.

PARA LA DEFENSA — la decisión sobre normalizar

Se probaron las dos versiones y el resultado no cambia, así que el TP usa los **datos crudos**. Eso es más defendible que normalizar porque sí, y ahorra tener que justificar la transformación. Normalizar **cambia el problema** —cambia qué significa «cerca»— y por lo tanto no es una decisión cosmética.

5.3 El problema real era la inicialización

Se corrió k -medias con $k = 3$ y **20 semillas distintas**, midiendo la inercia y la **pureza** (para cada grupo, cuántos patrones son de su clase mayoritaria, sumado y dividido por 148):

$$\text{pureza} = \frac{1}{L} \sum_i \max_j n_{ij}$$

Las 20 semillas caen en sólo **tres** soluciones:

Inercia	Pureza	Tamaños	Cuántas semillas
73,10	0,878	27 / 61 / 60	4
73,45	0,919	33 / 60 / 55	9
~116–117	0,709	22 / 88 / 38	7

13 de 20 llegan a inercia ~73; las otras 7 quedan atrapadas en ~116, que es donde cayó la semilla 2077. Mismo algoritmo, mismos datos: lo único que cambió fue por dónde arrancó. **El culpable era la inicialización**. Converge siempre en 1 a 6 iteraciones.

PARA LA DEFENSA — el hallazgo que no estaba planeado

La solución de **menor inercia (73,102)** no es la de **mayor pureza (0,919, con inercia 73,445)**. Es decir: **el acomodamiento que k -medias considera óptimo no es el que mejor coincide con las especies**. Tiene sentido — las especies las definió un botánico mirando plantas, la inercia es una cuenta de distancias en cuatro medidas; no tienen por qué coincidir, y no coinciden. Corolario práctico: hay que elegir la semilla **por menor inercia**, que es un criterio interno y no mira las etiquetas. Elegir la de mayor pureza sería usar las clases para entrenar, y eso rompe el carácter no supervisado del ejercicio.

Por qué el SOM no cayó en ese pozo con la misma semilla: el entorno arrastra a las neuronas mal inicializadas con sus vecinas de la rejilla. Es lo mismo que el punto 5 del ejercicio 1, donde sin entorno quedaban 79 de 100 neuronas muertas.

OJO — esto no contradice la conclusión del ejercicio 1

Allá el entorno **no** mejoraba la cuantización, y acá tampoco: es probable que k -medias tenga inercia parecida o menor. Lo que el entorno aporta es **robustez frente a la inicialización**, que es otra cosa. Y la aclaración honesta: con otra semilla k -medias también encuentra las tres especies. El problema es el arranque, no el método.

5.4 Por qué $k = 3$ y el SOM de 2×2

$k = 3$ porque Iris tiene 3 especies. En el SOM la rejilla es rectangular, así que con 3 neuronas la única forma posible es 1×3 — una **tira**, un mapa unidimensional — y el 2×2 es el mapa **bidimensional** más chico que existe. El costo de esa elección es que con 4 neuronas para 3 clases alguna clase tiene que partirse y la matriz queda de 3×4 .

Mapa	A favor	En contra
1×3	comparación pareja, matriz 3×3	es un SOM unidimensional
2×2	mapa bidimensional más chico	4 contra 3: una clase se parte
3×3	mapa 2D con lugar para ordenarse	9 contra 3, la matriz ya no se lee

Conviene mostrar los dos primeros y decir cuál se usó para qué. El mapa de 10×10 de los puntos 8 y 9 es bidimensional de todos modos, que es donde el enunciado sí pide explícitamente un mapa 2D.

5.5 El etiquetado del mapa grande

Los puntos 8 y 9 se resuelven con dos cuentas sobre el mapa de 10×10 :

- **Frecuencia de activación:** `np.bincount(ganadoras, minlength=cantidad_de_neuronas)`, con `minlength` para que las neuronas que nunca ganaron aparezcan con 0 en vez de desaparecer. Después `.reshape(filas, columnas)` convierte la lista plana en la cuadrícula — es `coordenadas_en_el_mapa` hecho de una sola vez para todas.
- **Clase de cada neurona:** de los patrones que ganó, la clase **más frecuente** (`argmax` del conteo). Las neuronas muertas quedan con -1 y sin clase.

PARA LA DEFENSA — el etiquetado es posterior y supervisado

El SOM se entrenó **sin ver una sola etiqueta**. Ponerle nombre a cada neurona pasa después, con los datos etiquetados, y es el procedimiento de la §8. Confundir las dos cosas es el error típico 3 del apunte de teoría.

6. Ejercicio 3 — Elegir k

El enunciado pide una métrica de clustering **de scikit-learn** y el k óptimo, probando entre 2 y 10.

La decisión de implementación: para cada k se corren **10 semillas** y **se toma la de menor inercia**. Sin eso, la curva mezclaría el efecto de k con el ruido de los mínimos locales que quedó demostrado en la sección 5.3 — y no se sabría qué se está mirando. Es lo que hace `sklearn` con `n_init`.

Las métricas y qué mide cada una:

Métrica	Qué compara	k óptimo
Inercia	distancia de cada patrón a su centroide	no tiene óptimo: siempre baja . Se busca el codo
Silueta	lo cerca que está cada patrón de su grupo contra el grupo vecino más cercano	el máximo
Calinski-Harabasz	dispersión entre grupos contra dispersión dentro	el máximo
Davies-Bouldin	cuán parecidos son los grupos más confundibles	el mínimo

La silueta va de -1 a $+1$: cerca de 1 significa que el patrón está cómodo en su grupo. Se promedia sobre todos.

OJO — la inercia sola no sirve para elegir k

Con $k = L$ (un centroide por patrón) la inercia da **cerro**. Siempre conviene más k , así que el mínimo no dice nada: por eso se busca el codo, el punto donde deja de bajar fuerte.

PARA LA DEFENSA — qué hacer si da $k = 2$

Con Iris pasa seguido, y **no es un error**. Dos de las tres especies se solapan tanto que, mirando sólo las cuatro medidas, la estructura más nítida son **dos** grupos: la especie separada por un lado, las otras dos juntas por el otro. Es exactamente lo que ya venían diciendo las matrices de contingencia del ejercicio 2. La conclusión en ese caso es que **el k que las métricas prefieren no coincide con la cantidad de especies**, porque las métricas miden geometría y las especies las definió la biología.

7. Errores típicos de esta implementación

1. **Calcular el entorno con aritmética de índices** ($\text{ganadora} \pm 1$, $\text{ganadora} \pm \text{columnas}$). La 69 y la 70 son consecutivas y están en esquinas opuestas: el mapa se «envuelve» y el ordenamiento sale mal sin que se note.
2. **Equivocar el axis**. $\text{axis}=0$ en las distancias devuelve un número por coordenada en vez de uno por neurona, y el SOM mueve siempre la misma. Se detecta imprimiendo `.shape`.
3. **Guardar el historial sin `.copy()`**. El GIF sale con todos los cuadros iguales.
4. **Invertir la resta en adaptar**. Con $\text{peso} - \text{patron}$ la neurona se aleja del dato y el mapa explota.
5. **Actualizar η y el radio por patrón en vez de por época**. Las duraciones del apunte están en épocas.
6. **Olvidar `minlength` en `bincount`**. Las neuronas muertas desaparecen del vector y la tabla queda corrida.
7. **Leer la contingencia como matriz de confusión**. La diagonal no significa nada: los números de grupo son arbitrarios.
8. **Elegir la semilla por pureza**. Usa las etiquetas para decidir, y rompe el carácter no supervisado.
9. **Decir que el SOM «clasifica»**. Agrupa; el etiquetado es posterior y supervisado.
10. **Presentar el reparto 20/20/60 como de la cátedra**. Es una decisión propia para comprimir las ~5000 épocas del apunte en las 1000 que permite el enunciado.

8. Las preguntas que conviene tener contestadas

1. ¿Qué es un peso en el SOM y en qué se diferencia del peso del perceptrón?
2. ¿Dónde está la cuadrícula en tu código? ¿Por qué el SOM unidimensional no necesitó cambiar el algoritmo?
3. ¿Por qué no calculás la raíz cuadrada en la distancia? ¿Cambiaría la ganadora?
4. ¿Dónde está la segunda rama de la ecuación de adaptación?
5. ¿Cuántas neuronas tiene el entorno con $R = 1$ si gana una de la esquina? ¿Programaste algún caso especial?
6. ¿De dónde salen los valores de las tres etapas? ¿Qué parte es de la cátedra y qué decidiste vos?
7. ¿Qué pasa si sacás el entorno? ¿Mejora o empeora el agrupamiento? ¿Y el orden?
8. ¿Por qué la T te deja neuronas sin usar y el círculo no?
9. ¿Por qué el SOM unidimensional cuantiza mejor la T que el bidimensional, y qué pierde a cambio?
10. ¿Por qué comparás con matrices de contingencia y no contando aciertos?
11. ¿Por qué k -medias te dio peor que el SOM con la misma semilla? ¿Es culpa del método?
12. ¿Normalizaste? ¿Por qué? ¿Qué cambió?
13. ¿En qué momento entran las clases de Iris, y por qué no antes?
14. Si la métrica del ejercicio 3 te da $k = 2$ y las especies son 3, ¿está mal el resultado?